Sales pipeline: design rationale

Product and scale

The screen helps 20 salespeople find, compare, select, and move 50,000 opportunities across seven stages. A table puts company, deal value, stage, owner, and activity into a consistent scanning order. Search, stage tabs, owner/inactivity filters, and sorting narrow the work before an action. JavaScript preserves the existing React/Vite setup and the agreed incremental scope.

A board was considered, but comparing thousands of opportunities and making bulk changes are easier in rows. Drag and drop adds precision and keyboard work without helping a 500-deal move, so explicit native stage controls are the primary action. Login, extra pages, and a real backend are outside this assignment.

Performance and selection

The fake server holds all records in memory. Client-side pages of 100, 250, or 500 bound each view; TanStack Virtual renders only a small window and overscan. TanStack Query manages loading, mutation lifecycle, and the cache. It does not replace virtualization. Unchanged record references and stable deal keys preserve existing rows during updates.

Selection is an explicit Set of deal IDs. The header checkbox affects the current page, including its offscreen rows. A separate action selects a snapshot of all matching IDs across pages. Changing filters never silently expands that selection. The toolbar shows total selection, selection on this page, and selection outside current filters. An implicit “all future matches” selection was rejected because a new search or teammate edit could change the scope of an already prepared action.

Saves and partial failure recovery

Moves update the visible stage immediately and show Saving. The fake API delays each single/bulk request by 300–1,500 ms and independently fails approximately 10% of eligible writes. Simulator controls can force success, force failure, or change delay. One bulk request returns per-deal results, avoiding thousands of timers and repeated copies of the full dataset.

Confirmed records and the latest local intent are separate. Failed moves remain visible as Unsaved. A virtualized recovery list identifies each ID, company, intended stage, confirmed stage, and error, independently of table filters. If 500 writes yield 480 successes and 20 failures, Retry sends those 20 IDs only. A batch-specific summary and retry action isolate them from earlier unrelated failures. Successful IDs are not rewritten or version-bumped again. Discard restores each failed deal's latest confirmed record; it never restores an old snapshot of the whole table. This preserves unrelated successful and pending edits. Selection clears on submission, while new selections made during saving are retained.

Concurrent edits and user context

Every write includes its expected confirmed version. The fake API checks that version atomically before writing and fences superseded delayed operations from this client. Responses advance confirmed data by version and change visible save status only for the current operation. An older response cannot overwrite newer intent.

Manual and automatic teammate events update authoritative records and notify subscribers. The client accepts only newer versions of known IDs. If a changed deal has a pending or failed local intent, the

intent remains visible as Conflict. Review compares the desired and current stages. Keep current stage accepts the teammate record; Save my stage starts an explicit version-checked write. Another intervening edit produces another conflict. Ordinary failure retries exclude conflicts.

Teammate updates change cells and summary totals immediately but preserve current row membership/order. Refresh view reapplies filters/sorting when the user chooses; explicit filter and local-edit actions also capture a new view. This deliberately allows a recently moved record to remain temporarily in its old filtered view rather than disappearing beneath a focused control. Selection, page, and scroll remain intact.

Accessibility, evidence, and limits

The UI uses a semantic table, total row counts/positions, labeled native controls, mixed page selection, visible focus, and live save summaries. Tab navigation mounts virtual rows before focusing them. Checkbox arrow/Home/End navigation and recovery focus keep keyboard users in context. Stage dropdown arrow behavior remains native.

Automated tests cover data integrity, page/selection boundaries, exact 480/20 partial results, targeted retry/rollback, stale responses, teammate conflicts, and resolution races. Browser checks cover large moves, bounded DOM rows, keyboard access, focus/scroll preservation, responsive layout, and recovery controls.

State and simulator settings reset on refresh. There is no real HTTP transport, durable storage, cross-tab synchronization, or production authorization. Summary totals include optimistic/unsaved values, with save status making that distinction visible. Next steps are server-side filtering/paging, durable idempotency and event ordering across clients, and broader assistive-technology testing.